

FARGO: Fast Maximum Inner Product Search via Global Multi-Probing

Xi Zhao¹, Bolong Zheng¹, Xiaomeng Yi², Xiaofan Luan², Charles Xie², Xiaofang Zhou³, Christian S. Jensen⁴

¹Huazhong University of Science and Technology, Wuhan, China ²Zilliz, Shanghai, China,

³Hong Kong University of Science and Technology, Hong Kong, China ⁴Aalborg University, Aalborg, Denmark

{zhaoxi,bolongzheng}@hust.edu.cn

{xiaomeng.yi,xiaofan.luan,charles.xie}@zilliz.com

zxf@cse.ust.hk, csj@cs.aau.dk

ABSTRACT

Maximum inner product search (MIPS) in high-dimensional spaces has wide applications but is computationally expensive due to the curse of dimensionality. Existing studies employ asymmetric transformations that reduce the MIPS problem to a nearest neighbor search (NNS) problem, which can be solved using locality-sensitive hashing (LSH). However, these studies usually maintain multiple hash tables and locally examine them one by one, which may cause additional costs on probing unnecessary points. In addition, LSH is applied without taking into account the properties of the inner product. In this paper, we develop a fast search framework FARGO for MIPS on large-scale, high-dimensional data. First, we propose a global multi-probing (GMP) strategy that exploits the properties of the inner product to globally examine high quality candidates. Companioned with GMP, we propose a global adaptive early termination condition that finds results quickly and offers theoretical guarantees. In addition, different with existing transformations that introduce either distortion errors or data distribution imbalances, we design a novel transformation, called random XBOX transformation (RXT), that avoids the negative effects of data distribution imbalances. Extensive experiments with real-world data offer evidence that FARGO is capable of outperforming existing proposals in terms of both accuracy and efficiency.

PVLDB Reference Format:

Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, Christian S. Jensen. FARGO: Fast Maximum Inner Product Search via Global Multi-Probing. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/Jacyhust/FARGO_VLDB23.

1 INTRODUCTION

Given a query point $q \in \mathbb{R}^d$ and a dataset $\mathcal{D} \subseteq \mathbb{R}^d$, the maximum inner product search (MIPS) problem aims to find a point $x \in \mathcal{D}$ that has the maximum inner product $q^\top x$. The MIPS problem in

high-dimensional spaces plays a vital role in a wide variety of applications, such as matrix factorization based recommendation [1, 34], multi-class label prediction [10, 21], similar-item retrieval [44], structural SVM [22] and deep learning [17], to name but a few.

A basic solution to the MIPS problem is the space partitioning tree-based method [23, 24, 36], which works well in low-dimensional spaces. However, when the dimensionality increases to several hundred or higher, the search time increases exponentially. This phenomenon is called the ‘curse of dimensionality’. *Since the MIPS problem is as complex as nearest neighbor search (NNS) [2, 5, 7], [For now, I have no idea about NNS and how complex it is. Besides, I think similar complexity is not the core to transform MIPS problem to NNS problem. You need to add more explanations instead of just a word ‘complex’, otherwise the ‘natural’ in the following sentence looks no natural at all.]* it is natural to consider the inner product as a similarity metric, and then apply *directly* [you say ‘directly’ here, but in the following, you begin to explain LSH cannot be applied directly, which makes me confused locality-sensitive hashing (LSH), one of the most popular tools used for find an approximate NN [3, 25, 27, 42, 47, 48]. One of requisites of applying LSH is that the point has the largest similarity to itself. However, the inner product does not satisfy this condition. For example, *a point x always exists that is different from q and such that $q^\top x > q^\top q$. [Give a concrete example, instead of saying the same thing once and again.]*

To address this issue, existing studies adopt asymmetric transformations for converting the MIPS into NNS. *[Following the logic of writing, it looks like that we should describe existing transformations and their drawbacks first (instead of LSH as below). If you adopt existing transformations directly, then the issue described above is not a challenge at all. Maybe move the next para here? BTW, by far, it is difficult for me to get the challenges in MIPS? List them 1. 2. 3.]* Specifically, they employ the basic (K, L) -bucketing to construct hash tables, and they use Multi-Probe for point probing that maintains a local probing sequence for each hash table and examine sequentially the hash tables. However, this may cause additional costs on probing unnecessary candidate points and affect the query performance. We develop a fast search framework FARGO for MIPS problem on large-scale, high-dimensional dataset. FARGO incorporates an accurate and efficient global multi-probing strategy (GMP) that generates a global probing sequence from all L hash tables. To improve the quality of candidates, we exploit the strong relationship between the inner product and a so-called quantization distance between the query point and a candidate point. In

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Summary of Notations

Notation	Definition
$\mathcal{D}$	Dataset of points in $\mathbb{R}^d$
$n = \mathcal{D} $	Dataset cardinality
d	Dimensionality
x	A point in $\mathcal{D}$
$h(x)$	Hash functions

addition, existing studies adopt an adaptive early termination condition for the approximate NNS problem [14, 26, 29, 30] to trade-off between accuracy and efficiency. However, existing adaptive early termination strategies are either time-consuming on training or not well-adapted for solving the MIPS problem. Therefore, we develop a novel and training-free global adaptive early termination strategy that enables a better trade-off between accuracy and efficiency.

There exist three main asymmetric transformation methods: (1) the L2-Transformation [38], (2) the Correlation-Transformation [39], and (3) the XBOX-Transformation [4, 45]. However, each of these transformations suffers from shortcomings that impact their performance negatively. Thus, the L2-Transformation and Correlation-Transformation introduce distortion errors. The XBOX-Transformation avoids distortion errors, but brings data distribution imbalances to the transformed dataset. In FARGO, we develop a novel asymmetric transformation, called random XBOX-Transformation (RXT), that maps the points in two different directions randomly so that the points are distributed more evenly.

The major contributions of FARGO are as follows:

- We propose an efficient and accurate method called GMP to answer MIPS queries that exploits the properties of the inner product to examine high quality candidates. In addition, we develop a global adaptive early termination that determines its termination conditions adaptively.
- We develop a novel asymmetric transformation method RXT that reduces data distribution imbalance.
- We conduct an extensive performance study using real datasets that covers the state-of-the-art methods for MIPS, finding that FARGO is efficient as well as accurate in terms of both the overall ratio and recall.

The rest of the paper is organized as follows. Section 2 presents the problem setting and the system overview of FARGO. Section ?? introduces the random XBOX-Transformation. Section 4 explains the construction and query algorithms of FARGO. Section 7 covers experimental studies that offer insight into the performance of the proposed FARGO and the main competitors. Section 8 reviews related work. Finally, Section ?? concludes the paper.

2 PRELIMINARIES

In this section, we introduce the preliminary knowledge. Frequently used notation is summarized in Table 1.

2.1 Problem Definition

DEFINITION 1 (MAXIMUM INNER PRODUCT SEARCH). Given a dataset $\mathcal{D} \subseteq \mathbb{R}^d$ that contains n points (a.k.a. vectors) and a query

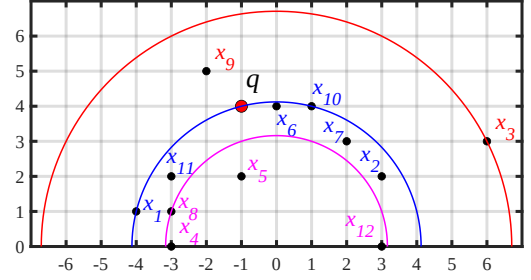


Figure 1: A Running Example

point $q \in \mathbb{R}^d$, the maximum inner product search (MIPS) returns a point $p \in \mathcal{D}$ that has the maximum inner product with q , i.e.,

$$p = \arg \max_{x \in \mathcal{D}} q^\top x. \quad (1)$$

The MIPS problem can be transformed to the nearest neighbor search (NNS) problem that aims to find a point x closest (most similar) to q . For example, the NNS problem in terms of Cosine similarity is as follows:

$$p = \arg \max_{x \in \mathcal{D}} \frac{q^\top x}{\|q\| \|x\|} = \arg \max_{x \in \mathcal{D}} \frac{q^\top x}{\|x\|}. \quad (2)$$

It is well known that finding the exact NN in high-dimensional space is inherently computational expensive due to the ‘curse of dimensionality’. A more attractive and generally sufficient alternative is to find an approximate nearest neighbor. In this paper, we focus on the transformation that converts an approximate maximum inner product search (c-MIPS) to an approximate nearest neighbor search (c-ANNS). We define them formally as follows:

DEFINITION 2 (c-APPROXIMATE NEAREST NEIGHBOR SEARCH). Given a query point $q \in \mathbb{R}^d$ and an approximation ratio $0 < c < 1$, let $\text{sim}(\cdot)$ be the similarity metric and $x^* \in \mathcal{D}$ be the exact nearest neighbor of q . A c -approximate nearest neighbor search (c-ANNS) returns a point $x \in \mathcal{D}$ satisfying $\text{sim}(x, q) \geq c \cdot \text{sim}(x^*, q)$.

DEFINITION 3 (c-MAXIMUM INNER PRODUCT SEARCH). Given a query point $q \in \mathbb{R}^d$ and an approximation ratio $0 < c < 1$, let $x^* \in \mathcal{D}$ be the point that has the maximum inner product with q . A c -maximum inner product search (c-MIPS) returns a point $x \in \mathcal{D}$ satisfying $q^\top x \geq c \cdot q^\top x^*$.

Without confusion, we use MIPS and c-MIPS interchangeably in the following.

EXAMPLE 1. Fig. 1 shows a query point q and a dataset with 12 points. x_9 is the exact MIPS of q with $q^\top x_9 = 22$. When $c = 0.5$, the c -MIPS returns any point in $\{x_6, x_9, x_{10}, x_{11}\}$ because the inner product is larger than $0.5 \times 22 = 11$.

2.2 Asymmetric Transformation

Given that the definitions of c-MIPS and c-ANNS are similar, the idea of considering the inner product as a similarity metric and then applying locality sensitive hashing (LSH), one of the most popular tools for c-ANNS, to solve c-MIPS is natural. However, a study [38] shows that the classical LSH framework is restrictive when solving MIPS. The existing proposals adopt an asymmetric transformation

that maps the data points from the original space to a transformed space, then apply LSH to return the c -ANN in the transformed space as the results of the c -MIPS. For better understanding, we briefly introduce an existing asymmetric transformation method, called XBOX-Transformation [19, 23, 33, 45], that consists of a function $P : \mathbb{R}^d \mapsto \mathbb{R}^{d+1}$ for data points and a function $Q : \mathbb{R}^d \mapsto \mathbb{R}^{d+1}$ for query points:

$$P(x) = [x; \sqrt{M^2 - \|x\|^2}], \quad (3)$$

$$Q(q) = [q; 0], \quad (4)$$

where $[\cdot]$ is the concatenation and $M = \max_x \|x\|$. Then, the Cosine similarity and Euclidean distance between $P(x)$ and $Q(q)$ can be computed as follows:

$$\text{sim}(P(x), Q(q)) = \frac{q^\top x}{\|q\| \cdot M}, \quad (5)$$

$$\text{dist}(P(x), Q(q)) = \|P(x), Q(q)\|^2 = \|q\|^2 + M^2 - 2q^\top x. \quad (6)$$

Explain more about how XBOX works Unlike the other transformation methods, such as L2-Transformation [38] and Correlation-Transformation [39] whose distances in the transformed space are approximations, XBOX-Transformation is an exact transformation, since both the Cosine similarity and Euclidean distance between $P(x)$ and $Q(q)$ can be computed as the inner product between x and q without distortion.

2.3 Locality Sensitive Hashing

Locality Sensitive Hashing (LSH) is a family of hash functions that enables c -ANNS in sublinear time with constant probability. An important property of LSH is that points with larger similarity have higher collision probability on their hash values.

DEFINITION 4 (LOCALITY SENSITIVE HASHING). *Given a similarity S_0 , an approximation ratio $0 < c < 1$, probability values p_1 and p_2 , where $p_1 > p_2$, a family $\mathcal{H} = \{h : \mathbb{R}^d \rightarrow U\}$ is called (S_0, cS_0, p_1, p_2) -locality sensitive, if for any $x, y \in \mathbb{R}^d$, it satisfies the following two conditions:*

- (1) If $\text{sim}(x, y) \geq S_0$ then $\Pr[h(x) = h(y)] \geq p_1$
- (2) If $\text{sim}(x, y) \leq cS_0$ then $\Pr[h(x) = h(y)] \leq p_2$

LSH is a generic framework and two LSH families for different similarity metrics are adopted widely.

- (1) One study [6] proposes an LSH family, called signed random projection (SRP), for Cosine similarity, which is defined as follows.

$$h(x) = \text{sign}(a^\top x), \quad (7)$$

where a is a random vector with each dimension $a_i \sim N(0, 1)$.

- (2) Another study [9] proposes an LSH family, called fixed window w random projection (w -RP), for Euclidean distance, which is defined as follows.

$$h(x) = \lfloor \frac{a^\top x + b}{w} \rfloor, \quad (8)$$

where a is a random vector with each dimension $a_i \sim N(0, 1)$, b is a real number uniformly drawn from $[0, w]$, and w is a user-specified window size.

After the transformation, a c -MIPS query is converted to a c' -ANNS query that can be solved using existing LSH methods. Note that $c \neq c'$ and the approximation ratio c' is a key factor that determines the difficulty of answering the c' -ANNS query, which has complexity $O(n^{1/c'})$ using the (K, L) -bucketing strategy [13, 15, 20]. Moreover, c' differs for similarity metrics, and a small c' incurs a high complexity. Given $c' = c_1$ for Cosine similarity and $c' = c_2$ for Euclidean distance, we demonstrate that c_1 is always larger than c_2 as follows.

DEFINITION 5 (ANGULAR DISTANCE). *For two data points x and y , the angular distance θ between x and y is defined as follows.*

$$\theta_{x,y} = \arccos \frac{x^\top y}{\|x\| \cdot \|y\|}. \quad (9)$$

For the ease of presentation, we use the angular distance and the Cosine similarity interchangeably when this does not cause ambiguity. Assume that $\|q\| = M$. Let x^* be q 's exact MIPS point, so $P(x^*)$ is $Q(q)$'s exact NNS point in the transformed space. Since $q^\top x^* < M^2$, we have $q^\top x^* = l \cdot M^2$, $0 < l < 1$. The c -MIPS query returns a point x where $P(x)$ is the result of the c' -ANNS query, so we have $q^\top x = c \cdot l \cdot M^2$. According to Eq. 3, we know that $\|P(x)\| = \|P(x^*)\| = M$.

For Cosine similarity, we have:

$$c_1 = \frac{\theta_{P(x), Q(q)}}{\theta_{P(x^*), Q(q)}} = \frac{\arccos \frac{q^\top x}{M \cdot M}}{\arccos \frac{q^\top x^*}{M \cdot M}} = \frac{\arccos(c \cdot l)}{\arccos(l)}.$$

For Euclidean distance, we have:

$$c_2 = \frac{\|P(x), Q(q)\|}{\|P(x^*), Q(q)\|} = \frac{\sqrt{2M^2 - 2c \cdot l \cdot M^2}}{\sqrt{2M^2 - 2 \cdot l \cdot M^2}} = \frac{\sqrt{1 - c \cdot l}}{\sqrt{1 - l}}.$$

It holds that $c_1 > c_2$ for any $c, l \in (0, 1)$. Therefore, we choose Cosine similarity as the metric and SRP as the LSH family for the c -ANNS in the transformed space.

3 FARGO INDEXING

FARGO aims to convert the c -MIPS problem into a c' -ANNS problem via the asymmetric transformation, and then solve the c' -ANNS problem by an LSH framework. Before transforming, we notice that a point with a large norm is more likely to be the c -MIPS result of a query point. Hence, we conduct a norm ranging strategy [19, 45] that divides the data points into disjoint partitions based on the norm.

3.1 Data Preprocessing

3.1.1 Data Partitioning. Intuitively, a large norm $\|x\|$ leads to a large inner product $q^\top x$ with a high probability, since $q^\top x$ is directly proportional to the norm $\|x\|$. Existing studies use so-called norm ranging to divide the data points into disjoint partitions, $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s$, according to their norms. We follow a strategy similar to that of an existing study [19] that uses two parameters b_0 and N_0 to bound the range of norms, where b is the norm ratio, and N_0 is the maximum number of points in each partition. The partitions satisfy three conditions:

- (1) For any two points $x \in \mathcal{D}_i$ and $y \in \mathcal{D}_j$, if $i < j$, we have $\|x\| \leq \|y\|$.

- (2) For any two points $x, y \in \mathcal{D}_i$, if $\|x\| \leq \|y\|$, we have $b_0 \cdot \|y\| < \|x\|$, $b_0 \in (0, 1)$.
- (3) For each partition $\mathcal{D}_i$, we have $|\mathcal{D}_i| < N_0$.

EXAMPLE 2. Assuming $N_0 = 6$ and $b = 0.7$, the dataset in Fig. 1 is divided into three partitions, as indicated by three different colors.

3.1.2 **Transformation.** After partitioning the data, we adopt the XBOX-Transformation in each partition $\mathcal{D}_i$ as follows,

$$P(x) = [x; \sqrt{M_i^2 - \|x\|^2}], \quad (10)$$

$$Q(q) = [q; 0], \quad (11)$$

where $M_i = \max_{x \in \mathcal{D}_i} \|x\|$ is the maximum norm of points in $\mathcal{D}_i$.

Compared to that in Eq. 3, the value of additional dimension in $P(x)$, $\sqrt{M_i^2 - \|x\|^2}$, can be smaller than that in $P(x)$, especially for the partitions that have a small norm. A smaller additional dimension is more beneficial for querying since it affects less on the distributions of transformed data. Consider the points x with $M \gg \|x\|$ in Eq. 3, their additional dimensions will be close to M and much larger than their original dimensions, which makes them have the similar hash values and hard to be differentiate by the hash functions. After the transformation using XT, we adopt SRP that maps all $P(x)$ in the transformed space into a projected space. For each hash function $h(\cdot)$, we randomly choose a vector a so that $h(P(x)) = \text{sign}(a^\top P(x))$. For simplicity, we denote $h(P(x))$ by $h(x)$. Then we use the (K, L) -bucketing strategy to build hash tables in the projected space.

3.2 Building Hash Tables

As mentioned, for each data point $P(x)$ in the transformed space, we adopt SRP as the LSH family so that $h(x) = \text{sign}(a^\top P(x))$. According to the property of SRP, we have the following.

LEMMA 1. For a query point q and a data point x , the angular distance between $Q(q)$ and $P(x)$ in the transformed space is:

$$\theta = \arccos \frac{Q(q)^\top P(x)}{\|Q(q)\| \cdot \|P(x)\|} = \arccos \frac{q^\top x}{\|q\| \cdot M}.$$

Without loss of generality, we assume that $\|q\| = 1$. Since M is a constant, the inner product of x and q is inversely proportional to the angular distance between $Q(q)$ and $P(x)$. The collision probability of $h(x)$ and $h(q)$ is computed as follows [33].

$$\Pr[h(x) = h(q)] = 1 - \frac{\theta}{\pi} \quad (12)$$

Next, we build hash tables using the (K, L) -bucketing strategy. After the norm ranging, for each partition $\mathcal{D}_i$, we build L hash tables and choose K hash functions $\mathcal{H} = \{h_1, \dots, h_K\}$ for each hash table. This yields $K \cdot L$ hash functions and 2^{KL} hash buckets for each partition. To simplify the computation and reduce the number of hash functions, all s partitions use the same $K \cdot L$ hash functions to build the index.

4 GLOBAL MULTI-PROBING

We proceed to introduce our query algorithm, GMP, for computing c-MIPS queries. To examine candidate points, we use a global multi-probe strategy to generate a probing sequence. Specifically, we evaluate the probability that a hash bucket B contains the nearest

neighbor via a so-called quantization distance (QD). The smaller the QD is, the higher the probability that B contains the nearest neighbor is. We determine which bucket to probe next among all $L \cdot 2^K$ buckets based on the global probing sequence.

4.0.1 **Overview of the Global Multi-Probing.** In the query phase, we first compute $Q(q)$ based on Eq. 18 and then project $Q(q)$ with the K hash functions for each hash table. Since the dataset is divided into s partitions, we examine them in descending order of the maximum norm. To generate a global probing sequence, we evaluate the probability that a hash bucket B contains the nearest neighbor via a so-called quantization distance (QD). The smaller the QD is, the higher the probability that B contains the nearest neighbor is. For each partition $\mathcal{D}_i$, we determine which bucket to probe next among all $L \cdot 2^K$ buckets based on the global probing sequence. This process ends when a termination condition is satisfied.

We address three important aspects of the GMP:

- (1) Why is QD an effective indicator for probing hash buckets in all hash tables? (see Section 4.1)
- (2) How to efficiently find the next hash bucket to probe among all unseen hash buckets? (see Section 4.2)
- (3) How to set a proper termination condition to balance the accuracy and the efficiency? (see Section 4.3)

4.1 Quantization Distance

Let $\zeta(x)$ be the projected vector of $P(x)$ under K hash functions, where $h_i(x) = \text{sign}(\zeta_i(x))$ and $\zeta_i(x) = a_i^\top P(x)$, $1 \leq i \leq K$. The quantization distance is defined as follows.

DEFINITION 6 (QUANTIZATION DISTANCE). For a query point q and a bucket $B = (b_1, b_2, \dots, b_K)$, the quantization distance between them is computed as follows.

$$QD(q, B) = \sum_{h_i(q) \neq b_i} |\zeta_i(q)|^2, 1 \leq i \leq K \quad (13)$$

$QD(q, B)$ is a random variable because it is the sum of a set of random variables $\zeta_i(q)$. Letting $B(x)$ be the hash bucket of a data point x , we use the cumulative distribution function $\Pr[QD(q, B(x)) \leq \omega]$ to describe the distribution of $QD(q, B(x))$. Note that the QD between q and $B(x)$ is computed by using the same K hash functions in the same hash table.

Let $\theta_{P(x), Q(q)}$ be the angular distance between $P(x)$ and $Q(q)$.

$$\theta_{P(x), Q(q)} = \arccos \frac{Q(q)^\top P(x)}{\|Q(q)\| \cdot \|P(x)\|}$$

We use the angular distance and the Cosine similarity interchangeably when this does not cause ambiguity. To show that QD is legal for ranking buckets from different hash tables, we first establish that $\Pr[QD(q, B(x)) \leq \omega]$ is only dependent on q and x .

LEMMA 2. Given a data point x with the angle $\theta_{P(x), Q(q)} = \theta$ between q and x in the transformed space, the cumulative distribution function of $QD(q, B(x))$ is determined only by θ and is independent on the hash table of $B(x)$.

For simplicity, we denote the cumulative distribution function of QD as follows.

$$\phi(\omega; \theta) = \Pr[QD(q, B(x)) \leq \omega] \quad (14)$$

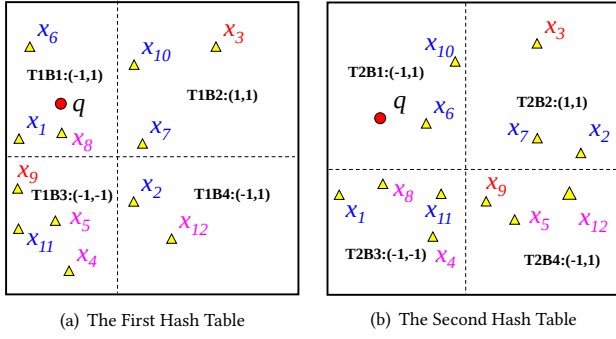


Figure 2: Point Projection on Two Hash Tables

LEMMA 3. $\varphi(\omega; \theta)$ monotonically decreases with θ . In other words, for $0 \leq \theta_1 < \theta_2 \leq \pi$ and $\omega > 0$, we have $\varphi(\omega; \theta_1) > \varphi(\omega; \theta_2)$.

Lemma 3 indicates that the smaller $\theta_{P(x), Q(q)}$ is, the smaller $QD(q, B(x))$ is likely to be. Lemmas 2 and 3 reveal that the quantization distance is an effective indicator for global probing in FARGO. The proofs of Lemmas 2 and 3 are provided in Section 6.1.

4.2 Generating Global Probing Sequence

Unlike Multi-Probe [31] that maintains a local probing sequence for each hash table, GMP generates a global probing sequence from all L hash tables based on Lemmas 2 and 3. We illustrate the advantage of GMP over Multi-Probe as follows.

EXAMPLE 3. Fig. 2 shows two hash tables after the projection by using SRP. Assume that for the first hash table, we have $a_{1,1} = (1.0, 0.1, -0.8)$, $a_{2,1} = (0.2, 0.9, -1.3)$. For the second hash table, we have $a_{1,2} = (0.6, -0.6, 1.1)$ and $a_{2,2} = (0.4, 0.9, -1.2)$. The projected values of q are $(\zeta_{1,1}(q), \zeta_{2,1}(q)) = (-1.6, 2.4)$ and $(\zeta_{1,2}(q), \zeta_{2,2}(q)) = (-3.0, 2.6)$. We denote by $TiBj$ the j -th bucket in the i -th hash table. The QD between q and each hash bucket is shown in Fig. 3(a). By examining the same number of candidate points (excluding the repeated points), say 8 points, we compare the accuracy of GMP and Multi-Probe. In Multi-Probe, we examine 4 candidate points in each hash table based on the local probing sequence in 3(c). In GMP, we examine 8 candidate points based on the global probing sequence in 3(b). We indicate the examined buckets with red color. In GMP, we examine T1B1, T2B1, T1B2, and T1B3, and we return x_9 . In Multi-Probe, we examine T1B1, T1B2, T2B1, T2B3, and T2B2, and we return x_6 . This example shows that GMP is able to achieve a higher accuracy than Multi-Probe when examining the same number of points.

Obviously, it is expensive to compute the QDs of all $2^K \cdot L$ buckets to q and sort them to construct a complete global probing sequence. Fortunately, by maintaining a small-sized heap, we are able to obtain efficiently the bucket with the least QD among all unseen buckets.

Let $\zeta_{i,j}(q)$ be the i -th entry of the projected vector in the j -th hash table. For a bucket $B = (b_1, \dots, b_K)$, from Eq. 13, we know that only the entries $\zeta_{i,j}(q)$ that have $h_{i,j}(q) \neq b_i$ contribute to the QD between q and B . Therefore, we can represent B by a set $S^j(B)$ that only records the indexes of the entries that differ from those of $h_{i,j}(q)$.

$$S^j(B) = \{i \mid b_i \neq h_{i,j}(q), 1 \leq i \leq K\}$$

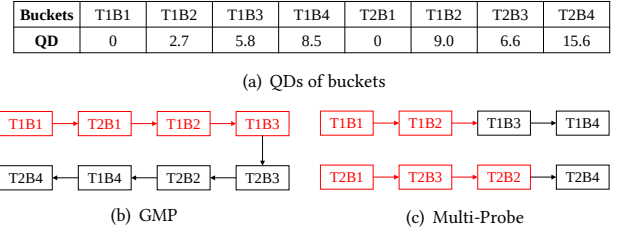


Figure 3: Probing Sequences of GMP and Multi-Probe

We reformulate Eq. 13 as follows.

$$QD(q, B) = \sum |\zeta_{i,j}(q)|^2, i \in S^j(B) \quad (15)$$

For simplicity, given the projected vector of q , we compute a sorted projected vector based on the value of $|\zeta_{i,j}(q)|$ so that we have $|\zeta_{i,j}(q)| < |\zeta_{i',j}(q)|$ for any $i < i'$ in the sorted projected vector. As a result, the smaller the entries in $S^j(B)$ are or the fewer entries $S^j(B)$ has, the smaller $QD(q, B)$ is. Note that by recording the indexes of the corresponding entries in the original projected vector of q , it is easy to reconstruct the hash bucket for a given set. Therefore, we proceed to generate a sequence of sets.

In the probing phase, we first probe the L buckets, where q is located in the L hash tables. We maintain a min-heap H that stores the candidate sets in ascending order of QD. Initially, we insert the set $S^j(B) = \{1\}$ into H for each hash table, since the hash bucket B that corresponds to $S^j(B) = \{1\}$ has the smallest QD to q in the j -th hash table. Next, we continue to process the top set of H and insert two sets into H that are generated by operations Shift() and Expand().

Let i_{max} be the maximum entry of the current top set $S^j(B)$.

- (1) Shift($S^j(B)$) replaces i_{max} by $i_{max} + 1$ in $S^j(B)$ and returns $S^j(B')$, which ensures that $S^j(B')$ is the set to generate that contains $S^j(B) \setminus \{i_{max}\}$ and that has the smallest QD.
- (2) Expand($S^j(B)$) inserts the entry $i_{max} + 1$ into $S^j(B)$ and returns $S^j(B')$, which ensures that $S^j(B')$ is the set to generate that contains $S^j(B)$ with the smallest QD.

When $i_{max} = K$, no new set is inserted into H . The Shift() and Expand() operations ensure the correctness of the generated probing sequence.

4.3 From c -MIPS to (I, c) -MIPS

After the transformation, a c -MIPS query is converted to a c' -ANNS query. An LSH method cannot solve a c' -ANNS problem directly, but can solve a threshold-based (θ, c') -ANNS problem for angular distance, which is defined as follows.

DEFINITION 7 ((θ, c')-ANNS). Assume a query point q , a dataset $\mathcal{D}$, an angle threshold θ , and an approximation factor $c' > 1$. The (θ, c') -ANNS query returns the following result:

- (1) If at least one point x exists in $\mathcal{D}$ such that $\theta_{q,x} < \theta$, it returns a point x' in $\mathcal{D}$ such that $\theta_{q,x'} < c' \cdot \theta$;
- (2) If no a point x exists in $\mathcal{D}$ such that $\theta_{q,x} < c' \cdot \theta$, it returns an empty set.

To conduct a c' -ANNS via LSH, we can conduct a sequence of (θ, c') -ANNS queries with increasing the angle θ . Similar to the (θ, c') -ANNS query, we define an (I, c) -MIPS query as follows.

DEFINITION 8 ((I, c)-MIPS). Assume a query point q , a dataset $\mathcal{D}$, an inner product threshold I , and an approximation factor $c < 1$. The (I, c) -MIPS query returns the following result:

- (1) If at least one point x exists in $\mathcal{D}$ such that $q^\top x > I$, it returns a point x' in $\mathcal{D}$ such that $q^\top x' > c \cdot I$;
- (2) If no point x exists in $\mathcal{D}$ such that $q^\top x > c \cdot I$, it returns nothing.

In a nutshell, to answer a c -MIPS query, we need to execute a sequence of (I, c) -MIPS queries while decreasing the inner product I . Each (I, c) -MIPS query corresponds to a (θ, c') -ANNS query that can be answered via LSH. The parameters θ and c' can be determined by a given I and c as follows.

$$\theta = \arccos\left(\frac{I}{M \cdot \|q\|}\right), \quad c' = \frac{\arccos\left(\frac{c \cdot I}{M \cdot \|q\|}\right)}{\arccos\left(\frac{I}{M \cdot \|q\|}\right)} \quad (16)$$

4.4 Termination Condition

Let T be the maximum number of points to probe and let t be a QD threshold. For a (θ, c') -ANNS query, the process terminates when one of the following conditions is satisfied:

- (1) We verify T candidate points;
- (2) With B^* being the bucket to probe next, $\text{QD}(q, B^*) > t$.

To ensure that FARGO returns correct (θ, c') -ANNS result, we determine the values of T and t as follows.

$$\begin{cases} (1 - \varphi(t; \theta))^L = 1/e \\ T = 2nL \cdot \varphi(t; c'\theta) \end{cases} \quad (17)$$

The intuition of Eq. 17 is that when the first condition is satisfied, we have found a correct (θ, c') -ANNS result with at least probability $1 - 1/e$; and when the second condition is satisfied, since there is at most $n \cdot \varphi(t; c'\theta)$ false positive points in a hash table, by using Markov's inequality, we have examined all false positive points in L hash tables with probability at least $1/2$. That is,

$$1 - \frac{1 - (1 - \varphi(t; c'\theta))^L}{2L \cdot \varphi(t; c'\theta)} > 1 - \frac{L \cdot \varphi(t; c'\theta)}{2L \cdot \varphi(t; c'\theta)} = \frac{1}{2}$$

We call such a termination condition as the normal termination condition (NT). NT is designed in a similar way as many mainstream LSH methods [9, 13, 42, 47], which guarantees to answer a c^2 -MIPS problem with a constant probability $1/2 - 1/e$ [13]. As mentioned, we answer a c^2 -MIPS query by conducting multiple (I, c) -MIPS queries. With x^* being q 's exact MIPS point with $q^\top x^* = I_{\max}$, we set $I = I_{\max}$ for the first (I, c) -MIPS query. For each (I, c) -MIPS query, we convert the (I, c) -MIPS query to a (θ, c') -ANNS query based on Eq. 16 and use NT to terminate the probing process. Once we find a point x with $q^\top x \geq cI$, we return it as the result. Otherwise, we decrease I to cI and conduct a (cI, c) -MIPS query.

5 IMPROVEMENTS

5.1 Random XBOX-Transformation

We develop a novel random XBOX-Transformation (RXT) to convert a c -MIPS query into a c' -ANNS query. Like the XBOX-Transformation

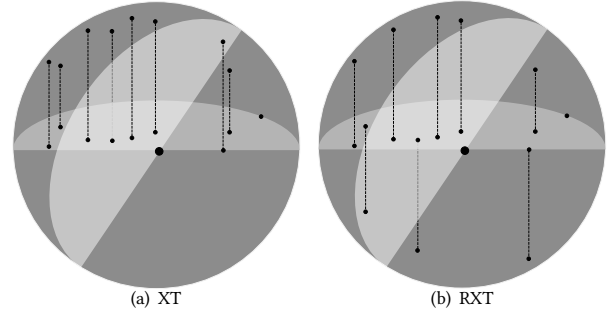


Figure 4: Comparison of RXT with XT

(XT), RXT has two functions $P : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$ and $Q : \mathcal{R}^d \mapsto \mathcal{R}^{d+1}$ for the partition $\mathcal{D}_i$, which are defined as follows.

$$\begin{aligned} P(x) &= [x; r \cdot \sqrt{M_i^2 - \|x\|^2}] \\ Q(q) &= [q; 0], \end{aligned} \quad (18)$$

where $r \in \{1, -1\}$ is a random variable and $\Pr[r = 1] = \Pr[r = -1] = \frac{1}{2}$. Note that RXT still satisfies Eq. 5. The difference of RXT from XT is that the sign of the $(d + 1)$ -st dimension of each transformed point $P(x)$ has 50% probability to be -1 . Fig. 4 shows a comparison between RXT and XT.

The point distributions in hash buckets are more balanced when using RXT than when using XT. We analyze this advantage by considering the probabilities of $h(x) = 1$ and $h(x) = -1$ when hashing the points.

LEMMA 4. Using XT, the number of points with hash value 1 is always different with the number of points with hash value -1 ; while using RXT, it is likely that the number of points with hash value 1 equals to the number of points with hash value -1 .

PROOF. For XT, we have $h(x) = \text{sign}(a^\top P(x)) = \text{sign}(\sum_{i=1}^d x_i a_i + \sqrt{M^2 - \|x\|^2} \cdot a_{d+1})$. Let $A = \frac{\sum_{i=1}^d x_i a_i}{\|x\|}$, we have $A \sim N(0, 1)$ according to the property of the sum of the normal distribution since $a_i \sim N(0, 1)$. Thus, $h(x) = \text{sign}(\|x\|A + \sqrt{M^2 - \|x\|^2} \cdot a_{d+1})$.

$$\Pr[h(x) = -1] = \Pr[A < -\frac{\sqrt{M^2 - \|x\|^2} \cdot a_{d+1}}{\|x\|}]$$

Consider the conditional probability $\Pr[h(x) = -1 \mid a_{d+1} = u] = \phi(-\frac{\sqrt{M^2 - \|x\|^2} \cdot u}{\|x\|})$, where $\phi(\cdot)$ is the cumulative distribution function of the standard normal distribution. If $u > 0$, then $-\frac{\sqrt{M^2 - \|x\|^2} \cdot u}{\|x\|} < 0$ and $\Pr[h(x) = -1 \mid a_{d+1} = u] < 1/2$ for any point x . In this case, it is likely that more than $n/2$ points have hash value 1. Likewise, if $u < 0$, it is likely that more than $n/2$ points have hash value -1 . For RXT, since $\Pr[r = 1] = \Pr[r = -1] = 1/2$, we have:

$$\begin{aligned} \Pr[h(x) = -1 \mid a_d + 1 = u] &= \phi\left(-\frac{r\sqrt{M^2 - \|x\|^2} \cdot u}{\|x\|}\right) \\ &= \frac{1}{2} \left[\phi\left(-\frac{\sqrt{M^2 - \|x\|^2} \cdot u}{\|x\|}\right) + \phi\left(\frac{\sqrt{M^2 - \|x\|^2} \cdot u}{\|x\|}\right) \right] = \frac{1}{2}. \end{aligned}$$

No matter what a_{d+1} is, it is expected that $n/2$ points have hash value 1, which completes the proof. $\square$

A hash function $h(\cdot)$ can be considered as a hyperplane that partitions the projected space into two sides. Lemma 4 implies that the points are more likely to be mapped to one side of the hyperplane using XT and that they are likely to be mapped evenly to the two sides of the hyperplane when using RXT. Therefore, RXT reduces the data distribution imbalance after the transformation, which benefits the subsequent point probing process.

5.2 Global Adaptive Early Termination

NT employs two parameters T and t that are fixed for all queries. However, the number of points to probe to reach a given accuracy varies across query points, which lowers the performance when facing data skew. Motivated by this, we develop a global adaptive early termination (AET) strategy that “adaptively” takes both the best found result and QD(q, B) into consideration. Assume that B^* is the next bucket to probe and I_0 is the currently found maximum inner product. We define two events according to I_0 and QD(q, B).

- **E1:** For a point x with $q^\top x \geq I_0/c$, the hash bucket in which x is located in a certain hash table remains unseen.
- **E2:** For a point x with $q^\top x \geq I_0/c$, in L hash buckets in which x is located, there is at least one that remains unseen.

When the probability that E2 happens, $\Pr[E_2]$, is small enough, the probability that we find a better result in the remaining buckets is small. It is then reasonable to terminate the query process. So we design AET by setting a failure probability p_τ as follows:

$$\begin{cases} \alpha_1 = \varphi(\text{QD}(q, B^*); \arccos(\frac{I_0}{c \cdot M_i \cdot \|q\|})) \\ \alpha_2 = 1 - (1 - \alpha_1)^L < p_\tau \end{cases} \quad (19)$$

Here, α_1 equals $\Pr[E_1]$ and $\alpha_2 = 1 - (1 - \Pr[E_1])^L$ equals $\Pr[E_2]$. Based on the intermediate query results, QD(q, B^*) and I_0 , AET verifies whether $\Pr[E_2]$ is within the allowed failure probability p_τ . When AET is satisfied, this indicates that the found results in $\mathcal{D}_i$ are correct results of the c -MIPS query. Compared to NT, the advantages of AET are:

- (1) The establishment of AET depends not on a fixed, predefined threshold T and t , but on the intermediate query results, which eliminates many unnecessary candidates, especially when the data is skewed;
- (2) AET is more efficient since we do not need to evaluate an initial $I_{\max}$ and conduct multiple (θ, c') -ANNS queries;
- (3) AET tightens the approximation ratio from c^2 to c and computes a c -MIPS query directly rather than a c^2 -MIPS query, which incurs a smaller query cost.

We describe how to compute a c -MIPS query using our solution that adopts AET as the termination condition in Algorithm 1.

With this condition, The probability that FARGO returns a correct c -MIPS result is at least $1 - \alpha_2$, which is bounded by $1 - p_\tau$. In practice, $p_\tau = 0.1$ suffices to provide good performance. GMP can be adapted easily to return k points with maximum inner products. We call the c -MIPS with k results (c, k) -MIPS. Specifically, to answer a (c, k) -MIPS query, we only need to set I_0 in Algorithm 1

Algorithm 1: Global Multi-Probing (GMP)

Input: A query point q , partitions $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s, c, p_\tau$, and L hash tables

Output: A point

```

1 Compute the hash values of  $q$ ;
2 Let  $x_0$  be the found best MIPS result;
3 for  $i = s \rightarrow 1$  do
4    $M_i \leftarrow \max_{x \in \mathcal{D}_i} \|x\|$ ;
5   if  $q^\top x_0 \geq cM_i$  then
6     Break;
7   while AET is not satisfied do
8      $B^* \leftarrow$  the bucket with the smallest QD in all the
       unseen buckets;
9     Verify the points in  $B^*$  and update  $x_0$ ;
10 return  $x_0$ ;
```

to the currently found k -th maximum inner product. We give an example of computing a $(0.5, 2)$ -MIPS query next.

EXAMPLE 4. In Fig. 1, we have $\mathcal{D}_1 = \{x_4, x_5, x_8, x_{12}\}$, $\mathcal{D}_2 = \{x_1, x_2, x_6, x_7, x_{10}, x_{11}\}$, and $\mathcal{D}_3 = \{x_3, x_9\}$. When a query q comes, we consider $\mathcal{D}_3$ first. We probe the hash tables according to the global probing sequence in Fig. 3(b) until the termination condition, AET, is satisfied. In this process, x_3 is found in T1B2, and x_9 is found in T1B3. Next, we consider $\mathcal{D}_2$. AET is satisfied after probing T1B2 and $I_0 = q^\top x_6 = 16$. Since $I_0 > 0.5 \cdot M_1$, we terminate the query process and return x_9 and x_6 .

Remark. AET is simpler and more efficient than existing adaptive termination strategies. Studies [14] and [26] train learning models to predict when the search terminates, which is time-consuming on training. However, AET is training-free and designed directly according to the property of quantization distance, which is easier to use without a large preprocessing time. One study [30] develops an adaptive termination condition on top of QA-LSH [18] by exploiting the best found result and current search radius, which is difficult to extend to other LSH methods. Moreover, it is not absolutely safe and may result in the probability of failure. In contrast, we provide a general adaptive early termination strategy. Once we obtain the distribution of the distance of q and x in the projected space, we can design an adaptive early termination strategy for an LSH method with the intermediate result.

6 THEORETICAL ANALYSIS

We proceed to provide a theoretical analysis of GMP. First, we present theoretical guarantees for GMP. Second, we derive the time and space complexity of the algorithm.

6.1 Theoretical Guarantees of GMP

6.1.1 Property of Quantization Distance. We prove Lemmas 2 and 3 that reveal that the quantization distance is an effective indicator for probing.

For a query point q and a data point x whose angle with q is $\theta \in (0, \pi)$, since QD($q, B(x)$) is the sum of a set of random variables, to determine the distribution of QD($q, B(x)$), we first analyze the

distribution of each random variable in $QD(q, B(x))$. Let $h_{i,j}$ be the i -th hash function in the j -th hash table. We define each item in $QD(q, B(x))$ as follows.

$$\mu_{i,j}(q, x) = \begin{cases} |\zeta_{i,j}(q)|^2, & h_{i,j}(q) \neq h_{i,j}(x) \\ 0, & h_{i,j}(q) = h_{i,j}(x) \end{cases}$$

LEMMA 5. Let ω be a non-negative number. The cumulative distribution function of $\mu_{i,j}(q, x)$ is:

$$\psi(\omega; \theta) = \Pr[\mu_{i,j}(q, x) \leq \omega] = 1 - \frac{\theta}{\pi} + \int_0^{\sqrt{\omega}} 2\phi(-u \cdot \cot \theta) d\phi(u),$$

where $\phi(x)$ is the cumulative distribution function of the standard normal distribution.

It is easy to prove Lemma 5 by exploiting the property of a normal distribution. Then, the probability density function of $\mu_{i,j}(q, x)$ can be written as follows via $\psi(\omega; \theta)$:

$$g(\omega; \theta) = \frac{\partial \psi(\omega; \theta)}{\partial \omega} = \begin{cases} \frac{1}{\sqrt{\omega}} \phi(-\sqrt{\omega} \cdot \cot \theta) f(\sqrt{\omega}) & \omega > 0 \\ (1 - \frac{\theta}{\pi}) \delta(\omega) & \omega = 0 \\ 0 & \omega < 0 \end{cases}$$

Here $\delta(\cdot)$ is the Dirac Delta function [16, 46], and $f(x)$ is the probability density function of the standard normal distribution.

We then prove Lemma 2.

PROOF. Given a point x , for any two different hash buckets in which x is located in these two hash tables. Since $\zeta_{i,j}(q)$ are independently and identically distributed, where $j \in \{1, 2\}$ identifies the table and $i \in [1, K]$, the distribution of $QD(q, B(x))$ is:

$$\Pr[QD(q, B(x)) \leq \omega] = \Pr[\sum_{i=1}^K \mu_{i,j}(q, x) \leq \omega] = \int_0^{\omega} g^{K*}(u; \theta) du,$$

where $g^{K*}(u; \theta)$ is the K -fold convolution of $g(u; \theta)$ [43] on u . This indicates that no matter which hash table $B(x)$ belongs to, $QD(q, B(x))$ has the same distribution. $\square$

Next, we prove Lemma 3.

PROOF. Let $\varphi_{(K)}(\omega; \theta)$ be the cumulative distribution function of $QD(q, B(x))$ when we have K hash functions. We prove Lemma 2 by induction. Let $P(K)$ be the statement that $\varphi_{(K)}(\omega; \theta)$ monotonically decreases with θ . We give the proof by induction on K .

Base case: The statement holds for the smallest positive integer number $K = 1$. Specially, when $K = 1$, $\varphi_{(1)}(\omega; \theta) = \psi(\omega; \theta)$. Further, we have:

$$\frac{\partial \psi(\omega; \theta)}{\partial \theta} = -\frac{1}{\pi} + \frac{1}{\pi} [1 - \exp(-\frac{\omega}{2 \sin^2 \theta})] < 0$$

So, $\psi(\omega; \theta)$ monotonically decreases with θ , and $P(1)$ is true.

Inductive step: We show that for any $K \geq 1$, if $P(K)$ holds, then $P(K+1)$ also holds. We have that:

$$\varphi_{(K+1)}(\omega; \theta_1) = \int_0^{\omega} g^{(K+1)*}(u; \theta_1) du$$

According to the definition of convolution, we have:

$$\begin{aligned} \int_0^{\omega} g^{(K+1)*}(u; \theta_1) du &= \int_0^{\omega} g(u; \theta_1) * g^{K*}(u; \theta_1) du \\ &= \int_0^{\omega} \int_0^u g(t; \theta_1) g^{K*}(u-t; \theta_1) dt du \end{aligned}$$

Interchanging the order of the integrals, we get:

$$\begin{aligned} &\int_0^{\omega} \int_0^u g(t; \theta_1) g^{K*}(u-t; \theta_1) dt du \\ &= \int_0^{\omega} g(t; \theta_1) dt \int_t^{\omega} g^{K*}(u-t; \theta_1) du \\ &= \int_0^{\omega} g(t; \theta_1) dt \cdot \varphi_{(K)}(\omega-t; \theta_1) \\ &> \int_0^{\omega} g(t; \theta_1) \varphi_{(K)}(\omega-t; \theta_2) dt = \int_0^{\omega} g(u; \theta_1) * g^{K*}(u; \theta_2) du \end{aligned}$$

Similarly, we have:

$$\begin{aligned} &\int_0^{\omega} g(u; \theta_1) * g^{K*}(u; \theta_2) du \\ &> \int_0^{\omega} g(u; \theta_2) * g^{K*}(u; \theta_2) du = \varphi_{(K+1)}(\omega; \theta_2) \end{aligned}$$

This shows that the statement $P(K+1)$ also holds true, establishing the inductive step.

Conclusion: Having shown the base case and the inductive step, statement $P(K)$ holds for every positive integer K . $\square$

6.2 Algorithm Analysis of FARGO

Assume that the dataset $\mathcal{D}$ is divided into partitions $\mathcal{D}_1, \dots, \mathcal{D}_s$. Given $n_i = |\mathcal{D}_i|$, we have $n_i = O(1)$, following Huang et al. [19]. Therefore, we set L and K as constants to compute MIPS on $\mathcal{D}_i$. To build the indexes in $\mathcal{D}_i$, we require computing $K \cdot L$ hash values for each point, which incurs $O(KLd)$ cost. Hence, the space consumption and indexing time of FARGO for $\mathcal{D}_i$ are $O(n_i)$ and $O(n_i d)$. Hence, FARGO has space cost $O(n)$ and indexing time $O(nd)$.

The query cost depends on how many false positive points we verify during the search. Assume that the query terminates at the bucket B^* and returns x with $q^\top x = I_0$, by using GMP strategy, we do not access any buckets B such that $QD(q, B) > QD(q, B^*)$. However, the Multi-Probe strategy cannot guarantee we do not access this kind of buckets, which enables it to always access more buckets than that with GMP strategy. For instance, if B^* is in the second hash table and x is found in the third table, the bucket B in the first and second hash tables are both likely to be accessed even though $QD(q, B) > QD(q, B^*)$ since AET does not satisfy. Assuming that the probability that a false positive point is verified is β , $\beta = 1 - (1 - \alpha)^L$ where $\alpha = \varphi(QD(q, B^*); \arccos(\frac{I_0}{M_i \cdot \|q\|}))$, and thus the point accessed is at most $O(\beta n)$. Although it is nearly impossible to simplify β due to the complex formation of φ , it is easy to find that $\beta < p_r$ since $\alpha < \alpha_1$. β is usually much smaller than 1 since $\varphi(t; \theta)$ decreases rapidly with θ and $\theta_1 = \arccos(\frac{I_0}{M_i \cdot \|q\|})$ is always much smaller than $\theta_2 = \arccos(\frac{I_0}{c \cdot M_i \cdot \|q\|})$. For instance, given $c = 0.5$, when $M_i = \|q\| = 1.0$ and $I_0 = 0.8$, $\theta_1 = 0.64$ and $\theta_2 = 1.16$. By setting $p_r = 0.1$, $\beta \approx 0.03$.

THEOREM 1. FARGO has space cost $O(n)$, indexing time $O(nd)$ and query cost $O(\beta nd)$, where β is much smaller than 1.

Table 2: Datasets

Dataset	$n (\times 10^3)$	d	Dataset	$n (\times 10^3)$	d
Audio	54	192	YahooMusic	625	300
MNIST	60	784	GIST	1,000	960
Cifar	60	1,024	Tiny5M	5,000	384
Trevi	100	4,096	Tiny80M	79,302	384

PROOF. Since L and K are constants, the space consumption is $O(n)$. The query time cost comes from two parts: 1) generating probing sequences; and 2) verifying the inner products between candidate points and q . In each partition $\mathcal{D}_i$, the former has cost $O(n_i \log n_i)$, and the latter has cost $O(\beta n_i d)$. Note that the probing sequence is the same in all partitions since we use the same $K \cdot L$ hash functions to build the index. So, we only have to generate the probing sequence once. Therefore, the total query time is $O(n_i \log n_i + \sum_{i=1}^s \beta n_i d) = O(\beta n d)$. $\square$

7 EXPERIMENTAL STUDY

We report on extensive experiments with real-world datasets that offer insight into the performance of FARGO. In particular, we aim to answer the following questions:

- Q1: Why is RXT a better transformation method than XT in FARGO?
- Q2: How does FARGO benefit from the adaptive early termination (AET) strategy?
- Q3: Why is GMP a better probing sequence than Multi-Probe?
- Q4: How does FARGO perform on the index size and indexing time relative to the competitors?
- Q5: How does FARGO perform on the query processing relative to the competitors under default settings?
- Q6: How does FARGO compare with the competitors in the Recall-Time dimensions and the Ratio-Time dimensions?

7.1 Experimental Settings

All algorithms are implemented in C++ and are compiled with the O3 optimization. All experiments are run on a Ubuntu Server with 2 Intel(R) Xeon(R) Gold 5218 CPUs and 254 GB RAM.

7.1.1 Datasets and Query Sets. We use 8 well-adopted real datasets varying in types, cardinality and dimensionality as shown in Table 2. Most of them are used in our competitor algorithms. Specifically, *YahooMusic* are often used in recommendation systems [5, 11, 24]. The other datasets are used widely in similarity search [19, 27, 47].

For queries, we randomly select 200 points from each dataset and repeat each experiment 20 times. We vary the value of k in $\{1, 10, 20, \dots, 100\}$ and set the default value to 50. We vary the value of c in $\{0.1, 0.2, \dots, 0.9\}$ and set the default value to 0.8.

7.1.2 Competing Algorithms. We have 6 state-of-the-art competitors for solving the c -MIPS problem:

- (1) **Simple-LSH** [33] uses the XBOX-Transformation and the (K, L) -bucketing strategy.
- (2) **Range-LSH** [45] applies norm-ranging strategy to partition the dataset and then employs Simple-LSH [33] in each partition.

- (3) **H2-ALSH** [19] uses the XBOX-Transformation and QALSH [18] after applying norm-ranging.
- (4) **RPT** [23] uses the XBOX-Transformation and randomized partition trees.
- (5) **ProMIPS** [40] projects high-dimensional data points to low-dimensional ones and accesses them by the ascending order of their distance to query point in the low-dimensional space.
- (6) **CeoMIPS** [35] chooses a few projections associated with the extreme values of the query signature and yields a sublinear query cost with search recall guarantee. Among three algorithms provided in [35], we adopts *CoCEOs* as our competitor.

7.1.3 Parameter Settings. FARGO needs to consider the parameters b_0 , N_0 , L , K , and p_τ . We set $K = 12$, $N_0 = 20480$, and $p_\tau = 0.1$ as defaults. Parameters b_0 and L are determined according to the data distribution. Typical values are $b_0 = \sqrt{0.95}$ and $L = 5$. For H2-ALSH, we set $c_0 = 2.0$ and $N_0 = 5000$ [19]. For ProMIPS, we are advised to set $p = 0.5$, $m = 10$, $k_p = 5$, $N_{key} = 40$ and $k_{sp} = 10$. For CeoMIPS, we set $D = 1024$, $s = 5$ and $B = n/10$. For the other three algorithms, we perform tuning to obtain parameter settings that achieve the best performance.

7.1.4 Evaluation Metrics. We use three performance metrics: query time (ms), overall ratio, and recall, where the query time evaluates efficiency and the overall ratio and recall evaluate result quality. For a query q , we denote the result of a (c, k) -MIPS query by $R = \{x_1, x_2, \dots, x_k\}$. Let $R^* = \{x_1^*, x_2^*, \dots, x_k^*\}$ be the exact k MIPS points. The overall ratio and recall are computed as follows.

$$OverallRatio = \frac{1}{k} \sum_{i=1}^k \frac{q^\top x_i}{q^\top x_i^*} \quad (20)$$

$$Recall = \frac{|R \cap R^*|}{|R^*|} \quad (21)$$

7.2 Self Evaluation

To evaluate the performance of FARGO, we first demonstrate that RXT and AET are better choices for the transformation method and termination condition than are XT and NT. Then, we compare the performance of FARGO with Multi-Probe.

7.2.1 Comparison of RXT and XT (Q1). To compare RXT and XT, we denote by FARGO-XT our solution with XT as the transformation method. For brevity, we only show the results on *MNIST*. We repeat the experiment 100 times and plot the findings in Fig. 5. We observe that FARGO is more stable than FARGO-XT in terms of both recall-time and ratio-time. In other words, FARGO achieves better accuracy in less time. Two observations explain this. (1) In each experiment, the data distribution after applying RXT is more even. (2) For the 100 experiments, FARGO generates similar data partitionings. That is, the index structure changes only little across the experiments.

7.2.2 Comparison of AET and NT (Q2). To compare the performance of AET and NT, we denote by FARGO-NT our solution with NT as the termination condition. We plot the recall-time and ratio-time curves on *MNIST* when varying p_τ for FARGO and varying T and t for FARGO-NT – see Fig. 6. We observe that FARGO takes less time than FARGO-NT to reach the same recall or overall ratio. A key reason is that AET adaptively decides when to terminate so

Table 3: Performance Overview

		FARGO	H2-ALSH	Simple-LSH	Range-LSH	RPT	ProMIPS	CeoMIPS
Audio	Query Time (ms)	2.060	4.145	8.826	3.110	4.411	3.174	2.632
	Recall	0.986	0.9636	0.9342	0.9762	0.9368	0.5554	0.8734
	Overall Ratio	0.9994	0.9985	0.9949	0.9985	0.9950	0.9119	0.9896
MNIST	Query Time (ms)	2.971	5.802	9.846	6.195	5.979	12.33	6.436
	Recall	0.8954	0.8772	0.5786	0.7014	0.5849	0.7836	0.2808
	Overall Ratio	0.9975	0.9966	0.9725	0.9864	0.9774	0.9892	0.9423
Cifar	Query Time (ms)	0.0947	0.6336	12.11	0.5984	4.4965	4.827	6.313
	Recall	0.9984	0.9974	0.9150	0.9694	0.9646	0.5498	0.909
	Overall Ratio	1.000	0.9999	0.9957	0.9994	0.9969	0.9645	0.9986
Trevi	Query Time (ms)	0.1863	3.809	27.73	3.819	15.71	27.09	-
	Recall	0.9996	0.9996	0.8458	1.000	0.7282	0.0048	-
	Overall Ratio	1.000	1.000	0.9961	1.000	0.9909	0.9059	-
YahooMusic	Query Time (ms)	28.89	35.51	115.3	33.06	58.57	119.0	31.1807
	Recall	0.9982	0.9958	0.5008	0.9928	0.4318	0.3736	0.9962
	Overall Ratio	0.9998	0.9997	0.6419	0.9986	0.5408	0.6340	0.9993
GIST	Query Time (ms)	0.4889	3.112	13.75	5.686	16.90	267.416	57.55
	Recall	0.9994	0.9986	0.2558	0.3724	0.4452	0.9692	0.5924
	Overall Ratio	1.000	0.9999	0.8447	0.9030	0.9125	0.9985	0.9575
Tiny5M	Query Time (ms)	0.6936	8.569	34.76	18.09	31.31	942.365	135.8
	Recall	0.972	0.9532	0.2062	0.6791	0.2754	0.7274	0.3306
	Overall Ratio	0.9997	0.9995	0.9361	0.9908	0.9558	0.9932	0.9722
Tiny80M	Query Time (ms)	12.54	21.17	231.2	36.45	412.6	17482	-
	Recall	0.8936	0.7646	0.1811	0.6764	0.4508	0.768	-
	Overall Ratio	0.9990	0.9836	0.9511	0.9913	0.9845	0.9960	-

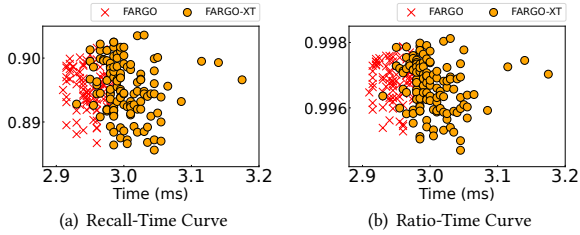


Figure 5: Comparison on RXT and XT

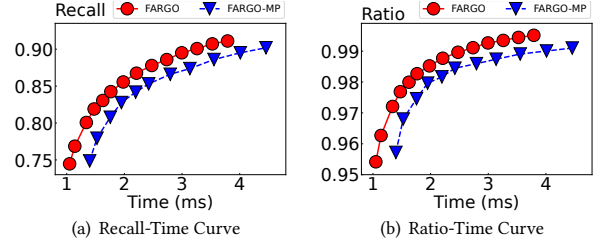


Figure 7: Comparison on GMP and Multi-Probe

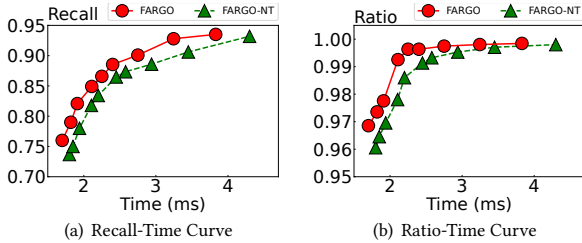


Figure 6: Comparison on AET and NT

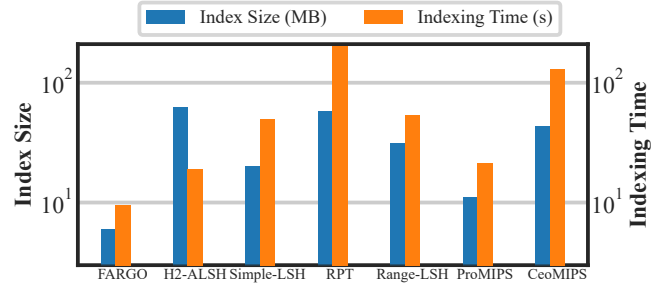


Figure 8: Indexing Performance on Gist

that fewer candidates are verified. Normally, to achieve a certain accuracy, the number of candidate points that need to be examined differs for each query point. NT does not consider this issue.

7.2.3 **Comparison of GMP and Multi-Probe (Q3).** To demonstrate that GMP is a better probing sequence than Multi-Probe, we

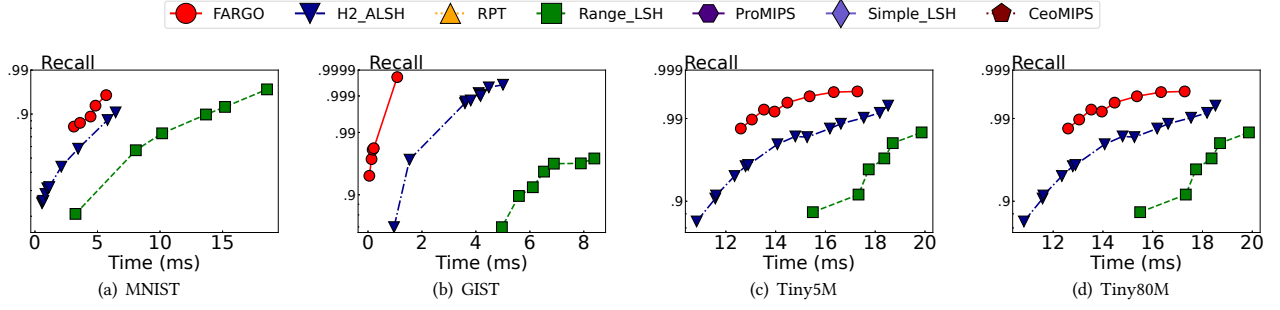


Figure 9: Recall-Time Curve

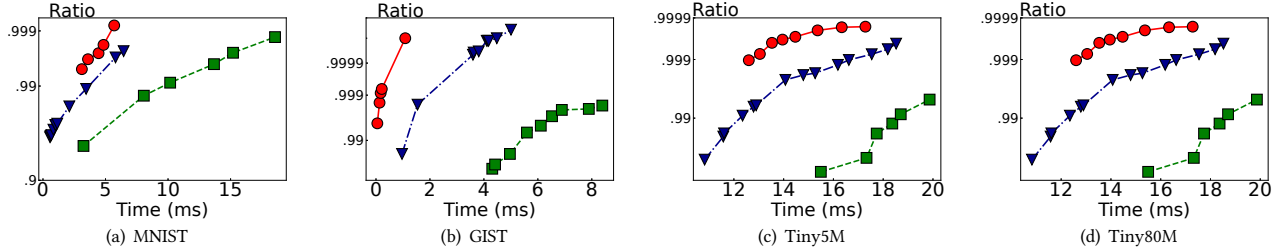


Figure 10: Ratio-Time Curve

denote by FARGO-MP our solution with Multi-Probe as the probing strategy. We compare the performance of FARGO and FARGO-MP with the same number of candidate points. By setting the number of candidate points T to a fixed value, FARGO examines T points according to the global probing sequence; FARGO-MP examines T/L points in each hash table according to each local probing sequence. We plot the recall-time and ratio-time curves on *MNIST* when varying T – see Fig. 7. We observe that FARGO takes less time than Multi-Probe to reach the same recall or overall ratio. This indicates that with the same number of candidate points, the results returned by FARGO are more accurate than those returned by FARGO-MP.

7.3 Comparison on Indexing Performance(Q4)

To evaluate the indexing performance of all algorithms, we compare their index size and indexing time on *Gist* with the default settings. The results are shown in Fig. 8. The index sizes of all algorithms except for ProMIPS depend directly on the number of indexes. FARGO only adopts $L = 5$ multi-dimensional hash tables and does not need to store the hash values, and thus has the smallest index size. ProMIPS adopts an m -dimensional indexes ($m = 10$) but requires storing m projected values of all points, which brings it a larger index size than FARGO. H2-ALSH adopts $m = 62$ hash indexes and has the largest index size. As for the indexing time, since FARGO, Simple-LSH and Range-LSH adopt hash tables as indexes, their indexing time of each index is the lowest. Their total indexing time depends on the number of indexes. FARGO achieves the lowest indexing time among all algorithms. H2-ALSH has a small indexing time because it just requires building indexes dimension by dimension. The indexing time of ProMIPS is also not high due to the small value of m . RPT and CeoMIPS have the much higher indexing time than other algorithms. The reason is that RPT

needs to build the multi-dimensional partition-based tree, which is more time-consuming than other structures. CeoMIPS needs to compute the extreme order statistics for $D = 1024$ projection functions, which incurs a extremely high computational cost.

7.4 Comparison on Query Performance(Q5, Q6)

7.4.1 Performance Overview. We report the query time (ms), overall ratio, and recall of all algorithms with default settings on all datasets in Table 3. FARGO has better query time than all competitors on all datasets, and the overall ratio and recall are also better than those of the competitors. We observe that *Cifar* and *GIST* incur the smallest query time. These datasets have nearly the same dimensionality but different cardinality, which indicates that the query time is not heavily affected by the dataset cardinality. This can be explained by the norm-ranging that eliminates the effect of cardinality. Another interesting observation is that the algorithms without norm-ranging, i.e., Simple-LSH, RPT, ProMIPS and CeoMIPS, have much higher query time than the algorithms with norm-ranging, i.e., FARGO, H2-ALSH, and Range-LSH, especially on large-scale dataset, such as *GIST*, *Tiny5M* and *Tiny80M*. This reveals that the norm-ranging plays an important role in MIPS. CeoMIPS fails to build indexes on *Trevi* and *Tiny80M*. The reason is that *Trevi* has many identical points and *Tiny80M* has a very large data size.

7.4.2 Effect of k . We study the query performance when varying the value of k in $\{1, 10, 20, \dots, 100\}$. For brevity, we only report the performance on datasets *GIST* – see Fig. 11. Since the query time is affected little by k and is similar as that at $k = 50$, we can find it in Table 3. FARGO achieves the best performance on all metrics. H2-ALSH is always second-best. These two algorithm nearly achieve the recall of 1.0 and overall ratio of 1.0. ProMIPS also

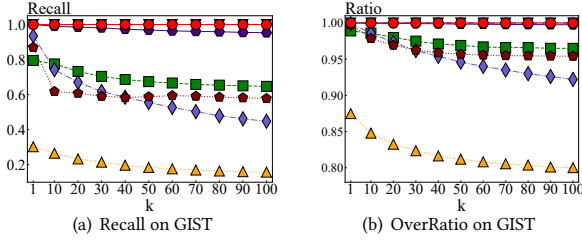


Figure 11: Query Performance when Varying k

achieves very high recall (> 0.96), but it costs much higher query time than FARGO and H2-ALSH (see Table 3). Therefore, its query performance is worse than these two algorithms. As k increases, all algorithms achieve smaller overall ratio and recall. This is because given nearly the same number of points checked, a higher k makes it more likely to miss some exact MIPS results.

7.4.3 Recall-Time and OverallRatio-Time Curves. We plot the recall-time and overall ratio-time curves for all algorithms on three datasets when varying c , L , and K – see Figs. 9 – 10. Since the algorithms without the norm-ranging performs much worse than FARGO, H2-ALSH, and Range-LSH, we do not compare them in this set of experiments. Moreover, the results of the remaining algorithms have been almost close to the exact MIPS. To make the performance difference more clear, we adopt the log-log plot to show the Recall-Time and OverallRatio-Time curves.

All algorithms return more accurate results with larger query times. FARGO again performs the best on all metrics, and H2-ALSH have comparable performance. On large datasets, the query time of the other algorithms increase rapidly with the cardinality of the dataset. This is due to the norm-ranging. We observe that the cardinality of *GIST* is 20 times of that of *MNIST*, while *GIST* incurs less query time than *MNIST*. This is because the points in *GIST* are distributed evenly so that it suffices to find good results in enough points with large norms. Among these three algorithms, FARGO performs better than H2-ALSH and Range-LSH. First, FARGO has a more accurate distance estimator than H2-ALSH and Range-LSH. Second, FARGO has a better transformation method and termination condition, which enables it to avoid examining unnecessary candidates when retrieved results are good enough.

8 RELATED WORK

The MIPS problem is a classic problem in the database community. Early methods employ space-partitioning trees, such as the M-tree [24], and the cone-tree [36]. However, these methods are computationally expensive in high-dimensional spaces due to the curse of dimensionality. Therefore, approximate solutions to the MIPS problem have been proposed [4, 23, 35, 38]. LSH is a mainstream approximation method for solving the MIPS problem. Since the inner product is not a metric, asymmetric transformation based methods are applied to convert the MIPS problem into NNS. Shrivastava et al. propose an L2-Transformation [38] and a Correlation-Transformation [39]. However, these two methods bring distortion errors and limit the query performance. XBOX-Transformation [4] eliminates the distortion errors and is widely adopted by many studies, such as Simple-LSH [33], Range-LSH [45], H2-ALSH [19],

RPT [23] and Ceos [35]. Based on the observation that a large norm $\|x\|$ is more likely to lead to a large inner product $q^\top x$, Range-LSH [45] and H2-ALSH [19] develop the norm-ranging strategy to partition the dataset according to their norm, which allows us to quickly probe the high-quality MIPS result in a small part of partitions and greatly improve the query performance. Next, many studies aim to terminate the query processing earlier based on the intermediate query status and demonstrate their effectiveness [26, 26, 29, 30, 40].

In addition, learning-based methods are proposed for MIPS [8, 12, 37]. Shen et al. [37] propose a binary code learning framework that preserves the inner product among raw data vectors. CeoMIPS [35] chooses a few projections associated with the extreme values of the query signature to answer MIPS problem. Next, neighbor graph-based methods are proposed for MIPS [28, 32, 41] that were originally used to solve approximate NNS problems. However, learning-based and graph-based methods both have extremely high construction cost, which is one or two order of magnitude higher than LSH methods and limits their usages on large-scale datasets.

REFERENCES

- [1] Mohamed Hussein Abdi, George Onyango Okeyo, and Ronald Waweru Mwangi. 2018. Matrix Factorization Techniques for Context-Aware Collaborative Filtering Recommender Systems: A Survey. *Sci* 11, 2 (2018), 1–10.
- [2] Thomas Dybdahl Ahle, Rasmus Pagh, Ilya P. Razenshteyn, and Francesco Silvestri. 2016. On the Complexity of Inner Product Similarity Join. In *PODS*. 151–164.
- [3] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya P. Razenshteyn. 2014. Beyond Locality-Sensitive Hashing. In *SODA*. 1018–1028.
- [4] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nave, and Ulrich Paquet. 2014. Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces. In *RecSys*. 257–264.
- [5] Robert M. Bell and Yehuda Koren. 2007. Lessons from the Netflix prize challenge. *SIGKDD Explorations* 9, 2 (2007), 75–79.
- [6] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.
- [7] Lijie Chen. 2018. On The Hardness of Approximate and Exact (Bichromatic) Maximum Inner Product. *Electronic Colloquium on Computational Complexity (ECCC)* 25 (2018), 26.
- [8] Xinyan Dai, Xiao Yan, Kelvin Kai Wing Ng, Jiu Liu, and James Cheng. 2020. Norm-Explicit Quantization: Improving Vector Quantization for Maximum Inner Product Search. In *AAAI*. 51–58.
- [9] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*. 253–262.
- [10] Thomas L. Dean, Mark A. Ruzon, Mark Segal, Jonathon Shlens, Sudheendra Vijayanarasimhan, and Jay Yagnik. 2013. Fast, Accurate Detection of 100, 000 Object Classes on a Single Machine. In *CVPR*. 1814–1821.
- [11] Gideon Dror, Noam Koenigstein, Yehuda Koren, and Markus Weimer. 2012. The Yahoo! Music Dataset and KDD-Cup '11. In *KDD Cup (JMLR, Vol. 18)*. 8–18.
- [12] Marco Fraccaro, Ulrich Paquet, and Ole Winther. 2016. Indexable Probabilistic Matrix Factorization for Maximum Inner Product Search. In *AAAI*. 1554–1560.
- [13] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.
- [14] Anna Gogolou, Theophanis Tsandilas, Karima Echihabi, Anastasia Bezerianos, and Themis Palpanas. 2020. Data Series Progressive Similarity Search with Probabilistic Quality Guarantees. In *SIGMOD*. 1857–1873.
- [15] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani. 2012. Approximate Nearest Neighbor: Towards Removing the Curse of Dimensionality. *Theory Comput.* 8, 1 (2012), 321–350.
- [16] Sadri Hassani. 2009. Dirac delta function. In *Mathematical methods*. Springer, 139–170.
- [17] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *WWW*. 173–182.
- [18] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search. *PVLDB* 9, 1 (2015), 1–12.
- [19] Qiang Huang, Guihong Ma, Jianlin Feng, Qiong Fang, and Anthony K. H. Tung. 2018. Accurate and Fast Asymmetric Locality-Sensitive Hashing Scheme for Maximum Inner Product Search. In *KDD*. 1561–1570.
- [20] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.
- [21] Prateek Jain and Ashish Kapoor. 2009. Active learning for large multi-class problems. In *CVPR*. 762–769.
- [22] Thorsten Joachims, Thomas Finley, and Chun-Nam John Yu. 2009. Cutting-plane training of structural SVMs. *Mach. Learn.* 77, 1 (2009), 27–59.
- [23] Omid Keivani, Kaushik Sinha, and Parikshit Ram. 2018. Improved maximum inner product search with better theoretical guarantee using randomized partition trees. *Machine Learning* 107, 6 (2018), 1069–1094.
- [24] Noam Koenigstein, Parikshit Ram, and Yuval Shavitt. 2012. Efficient retrieval of recommendations in a matrix factorization framework. In *CIKM*. 535–544.
- [25] Naama Kraus, David Carmel, Idit Keidar, and Meni Orenbach. 2016. NearBucketLSH: Efficient Similarity Search in P2P Networks. In *SISAP (Lecture Notes in Computer Science, Vol. 9939)*. 236–249.
- [26] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *SIGMOD*. 2539–2554.
- [27] Jinfeng Li, Xiao Yan, Jian Zhang, An Xu, James Cheng, Jie Liu, Kelvin Kai Wing Ng, and Ti-chung Cheng. 2018. A General and Efficient Querying Method for Learning to Hash. In *SIGMOD*. 1333–1347.
- [28] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and Improving Proximity Graph Based Maximum Inner Product Search. In *AAAI*. 139–146.
- [29] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, and Lu Qin. 2019. I-LSH: I/O Efficient c-Approximate Nearest Neighbor Search in High-Dimensional Space. In *ICDE*. 1670–1673.
- [30] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, Lu Qin, and Xuemin Lin. 2021. EI-LSH: An early-termination driven I/O efficient incremental c-approximate nearest neighbor search. *VLDB J.* 30, 2 (2021), 215–235.
- [31] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search. In *VLDB*. 950–961.
- [32] Stanislav Morozov and Artem Babenko. 2018. Non-metric Similarity Graphs for Maximum Inner Product Search. In *NeurIPS*. 4726–4735.
- [33] Behnam Neyshabur and Nathan Srebro. 2015. On Symmetric and Asymmetric LSHs for Inner Product Search. In *ICML*, Vol. 37. 1926–1934.
- [34] Jennifer Nguyen and Mu Zhu. 2013. Content-boosted matrix factorization techniques for recommender systems. *Stat. Analysis and Data Mining* 6, 4 (2013), 286–301.
- [35] Ninh Pham. 2021. Simple Yet Efficient Algorithms for Maximum Inner Product Search via Extreme Order Statistics. In *KDD*. ACM, 1339–1347.
- [36] Parikshit Ram and Alexander G. Gray. 2012. Maximum inner-product search using cone trees. In *KDD*. 931–939.
- [37] Fumin Shen, Wei Liu, Shaoting Zhang, Yang Yang, and Heng Tao Shen. 2015. Learning Binary Codes for Maximum Inner Product Search. In *ICCV*. 4148–4156.
- [38] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). In *NIPS*. 2321–2329.
- [39] Anshumali Shrivastava and Ping Li. 2015. Improved Asymmetric Locality Sensitive Hashing (ALSH) for Maximum Inner Product Search (MIPS). In *UAI*. 812–821.
- [40] Yang Song, Yu Gu, Rui Zhang, and Ge Yu. 2021. ProMIPS: Efficient High-Dimensional c-Approximate Maximum Inner Product Search with a Lightweight Index. In *ICDE*. IEEE, 1619–1630.
- [41] Shulong Tan, Zhixin Zhou, Zhaozhao Xu, and Ping Li. 2019. On Efficient Retrieval of Top Similarity Vectors. In *EMNLP/IJCNLP* (1). 5235–5245.
- [42] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2009. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD*. 563–576.
- [43] Christoph Haehling von Lanzener and William N Lundberg. 1974. The n-fold convolution of a mixed density and mass function. *ASTIN Bulletin: The Journal of the IAA* 8, 1 (1974), 91–103.
- [44] Mengshuang Wang, Jun Ma, Shanshan Huang, and Peizhe Cheng. 2015. Combining Positive and Negative Feedbacks with Factored Similarity Matrix for Recommender Systems. In *WAIM*, Vol. 9098. 233–246.
- [45] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *NeurIPS*. 2956–2965.
- [46] Lin Zhang. 2020. Dirac delta function of matrix argument. *International Journal of Theoretical Physics* (2020), 1–28.
- [47] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *PVLDB* 13, 5 (2020), 643–655.
- [48] Yuxin Zheng, Qi Guo, Anthony K. H. Tung, and Sai Wu. 2016. LazyLSH: Approximate Nearest Neighbor Search for Multiple Distance Functions with a Single Index. In *SIGMOD*. 2023–2037.